

IPMS WEB APPLICATION & SYSTEM ERROR RESOLUTION MANUAL

Document ID: DOC-2026-IPMS-ERR

Classification: Technical Support & Operations Guide

SECTION 1: PERFORMANCE ISSUES (LAGGING & HIGH LATENCY)

Error Category: System Performance & Network Bottlenecks

When operators or administrators experience noticeable sluggishness, delayed responses, or high latency metrics on the Operator Terminal (`localhost:5173`) or Admin Dashboard (`localhost:5174`), investigate the following root causes:

1. Local LLM Resource Saturation (Ollama):

- *Cause:* Running a local large language model simultaneously with heavy compilation or multiple frontends can max out GPU/CPU resources on the host machine (e.g., ASUS TUF A15 hardware limitations).
- *Resolution:* Check background processes to ensure Ollama isn't queuing multiple concurrent token-generation requests. Keep prompt lengths concise during high-load shifts.

2. Database Query Inefficiencies:

- *Cause:* Large volumes of unindexed logs in the `TelemetryLogs` SQLite table can slow down `GROUP BY` aggregation queries used for dashboard statistics.
- *Resolution:* Run periodic database maintenance or check the C# Entity Framework Core LINQ queries to ensure proper asynchronous execution (`ToListAsync()`).

3. Frontend Re-render Loops:

- *Cause:* Excessive state updates inside React components can cause unnecessary component re-renders, resulting in sluggish UI response times.
- *Resolution:* Use React Developer Tools to profile component lifecycles and ensure `useEffect` dependency arrays are properly optimized.

SECTION 2: UI FREEZING & UNRESPONSIVENESS

Error Category: Client-Side Application Crashes

If the web interface freezes completely or becomes entirely non-responsive during operation:

1. Browser Memory Leaks:

- *Cause:* Leaving the Admin Dashboard open for extended periods with continuous live-polling intervals can cause memory consumption to spike in Chrome/Edge.
 - *Resolution:* Refresh the application session. Verify that interval cleanups (`clearInterval`) are implemented inside React `useEffect` hooks.
2. **Recharts / DOM Overload:**
- *Cause:* Attempting to render thousands of data points simultaneously inside Recharts containers without slicing or aggregation.
 - *Resolution:* Ensure the analytics endpoints or frontend array methods slice data appropriately (e.g., restricting graphs to the top 5 or top 10 most frequent topics).
3. **Network Connection Drops to C# Backend:**
- *Cause:* If the C# Web API (`localhost:5268`) crashes or is accidentally terminated via the terminal, asynchronous `fetch` calls will hang or trigger unhandled promise rejections.
 - *Resolution:* Ensure backend terminal is active (`dotnet run`) and implement robust `try/catch` error states with user-friendly notification banners in the React frontend.

SECTION 3: AUTHENTICATION FAILURES (LOGIN ISSUES)

Error Category: Access Control & Security

Authentication errors prevent users from accessing secure terminal interfaces. Common failure scenarios include:

1. **Invalid Credentials Error:**
 - *Cause:* Entering mismatched usernames or hardcoded passwords (e.g., default admin credentials `admin / fgv-ipms-2026`).
 - *Resolution:* Verify environment configuration files (`.env`) or check frontend state validation logic.
2. **CORS & Port Mismatch Blocks:**
 - *Cause:* The React frontend attempting to communicate with an incorrect backend port (e.g., pointing to `5174` instead of `5268`), triggering browser CORS security blocks.
 - *Resolution:* Check browser Developer Tools (F12) under the **Console** and **Network** tabs to verify that API requests target `http://localhost:5268` successfully.
3. **SQLite Authentication Table Lock:**
 - *Cause:* Concurrent write operations locking the local database file during rapid login attempts.
 - *Resolution:* Ensure Entity Framework Core DbContext is configured correctly with proper dependency injection scopes (`AddScoped`).

SECTION 4: PASSWORD CHANGE & UPDATE ERRORS

Error Category: User Account Management

When operators or administrators attempt to update security credentials or modify user account settings:

1. **Old Password Mismatch Validation:**

- *Cause:* The system rejects password update requests if the verification string for the existing password fails hash comparisons.
- *Resolution:* Ensure the input form requires explicit confirmation fields and matches backend salt/hashing requirements.

2. **Constraint Violations in SQLite:**

- *Cause:* Attempting to update a user record with an existing unique identifier or submitting blank payload parameters.
- *Resolution:* Implement strict client-side form validation using Tailwind CSS error states to prevent empty string submissions to the C# API.